Project : Localizing iEEG contacts in a CT scan

LGBIO2050 Medical Imaging

Nicolas Delinte, Simon François & Pauline Hermans

1 Context

Approximately one third of patients with epilepsy are resistant to anti-epileptic drugs. One possible treatment for these patients is epilepsy surgery, which consists of disconnecting the brain regions responsible for triggering seizures. To help neurosurgeons precisely identify the onset region while minimizing the amount of brain tissue removed, intracranial electroencephalography (iEEG) can be used.
iEEG is a technique that records electrical activity directly from the brain using electrodes implanted within the brain. These electrodes consist of thin probes, several centimeters long, containing approximately 6 to 18 individual contact points, each recording electrical activity from a specific location.
To accurately localize the electrodes within the brain, CT scans are acquired after electrode implantation. Since the electrodes are visible on CT images, their positions can be identified and their 3D coordinates extracted. These coordinates can then be co-registered with anatomical MRI scans to precisely determine which brain regions are sampled by each electrode contact.

In this project, you will work with CT scans from epilepsy patients implanted with iEEG electrodes. The objective is to process the images to generate a clean brain mask and extract the 3D coordinates of the electrode contacts.

2 Data

CT scans

The CT scans used in this project are 3D images. Therefore, we no longer refer to pixels, but to voxels (volume elements). The files are provided in the NIfTI format (`.nii` or `.nii.gz`), a standard file format widely used in medical imaging. NIfTI files store volumetric data (3D or 4D), along with metadata such as voxel size, image orientation, etc. They can be easily handled in Python using libraries such as NiBabel.

An important feature of NIfTI files is the affine transformation matrix stored in their metadata. This is a 4×4 matrix that defines the relationship between voxel coordinates (i,j,k) and their corresponding positions in physical space (x,y,z). In other words, it specifies the spatial location, orientation, and scale of the image in the real world.

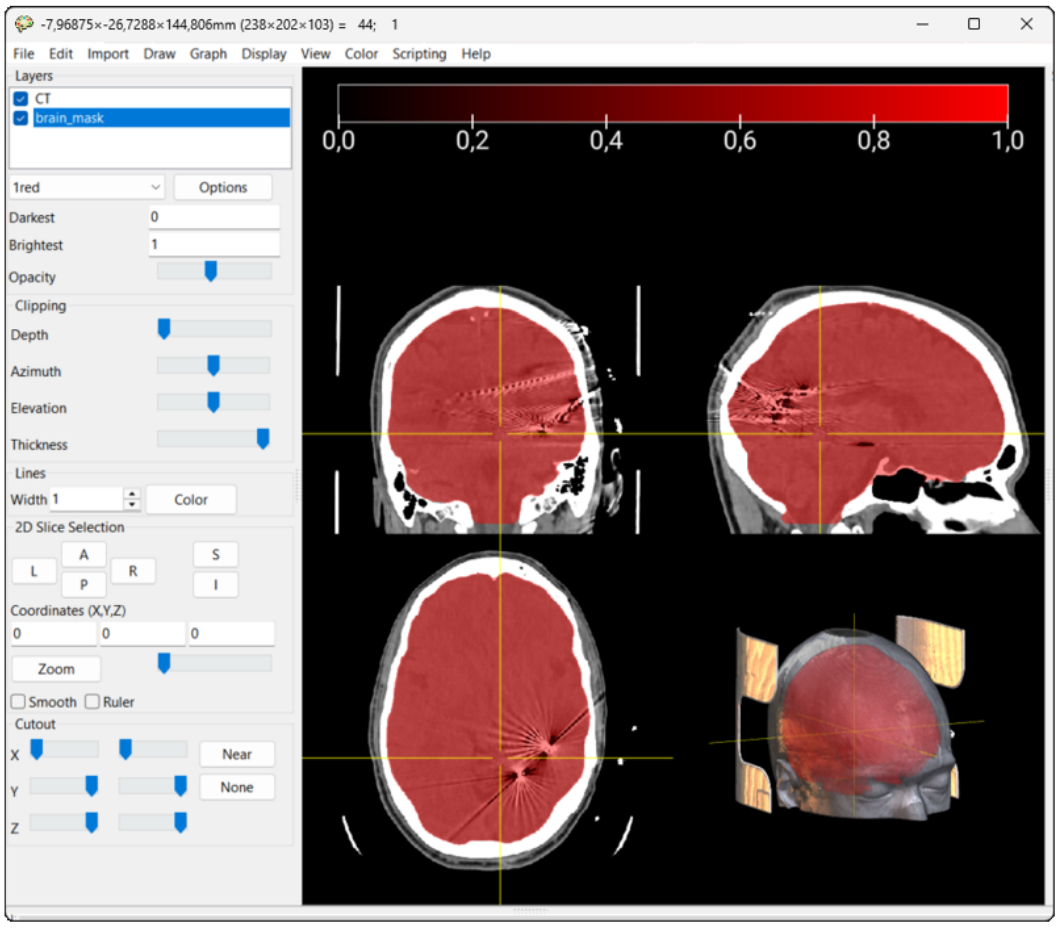
You will not need to manipulate this matrix for this project. However, it is important to preserve the affine information when creating or saving new NIfTI files. This ensures that different images remain correctly aligned in physical space and can be accurately overlaid. For example, a segmentation mask should be superimposed on the original CT scan at the correct location, with the same orientation and spatial dimensions.

To help you with this, we provide dedicated functions to load NIfTI files and save the resulting images while preserving the relevant spatial information and metadata, as well as some visualization functions.

Data visualization

Plotting 2D slices of the volumes is useful for inspecting the results, but exploring the data in three dimensions can provide an even better understanding of the anatomical structures, the spatial distribution of the electrodes, and the quality of your brain mask segmentation and electrode contact point detection.

Although this step is not mandatory, you are encouraged to explore your results interactively using MRIcroGL, a free, open-source medical image viewer. It allows you to load and visualize both the original CT volume and its corresponding segmentation mask, provided as `.nii.gz` files.



Quick MRIcroGL user guide
The software can be downloaded from this website.

Use the menu **File** → **Open** to load the CT volume (`.nii` or `.nii.gz`). Then, load the corresponding segmentation mask using **File** → **Add Overlay** and selecting the matching mask file.

MRIcroGL displays three synchronized slice views (axial, sagittal and coronal), alongside a 3D rendering pane. You should obtain a window similar to the one shown in the figure below. You can customize the views displayed using the **Display** menu.

3 Project tasks

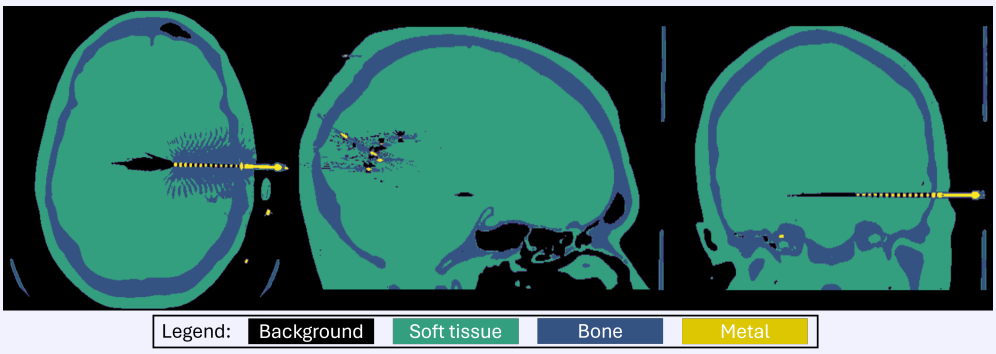
The project is divided into three successive tasks, each of which must be performed on all three subjects using the same code and parameter settings.

A. Preliminary segmentation

The first step is to segment the three main types of materials visible in the CT images: metal (electrodes), bone (skull), and soft tissue (brain, muscle, fat, skin, etc.). This initial segmentation should rely exclusively on pixel intensity values. Consider the physical properties of each material and how they influence its appearance in a CT image. How could these differences be used to distinguish between the three classes?

Output task A

A NIfTI file containing the segmentations (0 for background voxels, 1 for soft voxels, 2 for bone voxels, 3 for metal voxels).



B. Brain mask refining

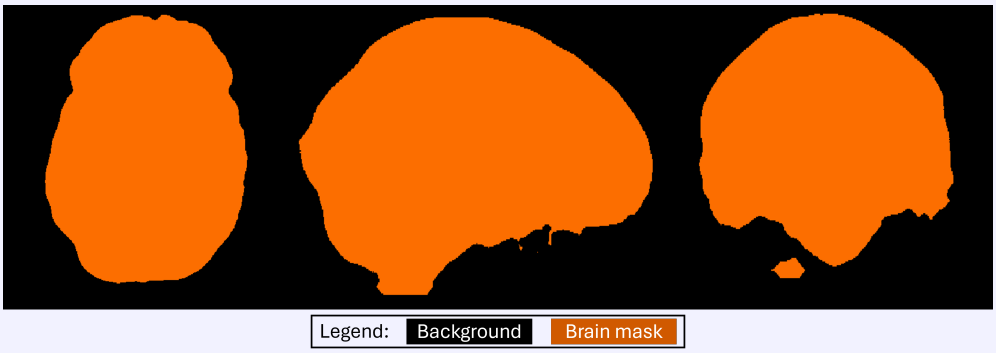
Due to the presence of image artifacts created by the electrodes, the soft tissue mask generated in task A contain imperfections. Furthermore, soft tissue includes not only the brain but also other structures such as the skin, muscles and other surrounding

tissues.

The objective of task B is therefore to use the soft tissue mask obtained in task A as a starting point to isolate the brain and generate a clean brain mask. The parts of the electrodes located inside the brain must be included in the final brain mask.

Output task B

A NIfTI file containing the brain mask as a binary segmentation (0 for background voxels, 1 for brain voxels).



C. Electrodes contacts coordinates extraction

The final step is to extract the coordinates of the electrode contacts located inside the brain. You are asked to isolate each individual contact point, reducing it to a single point, and determine its corresponding voxel coordinates (i,j,k). The contact points located outside of the brain mask should not be included.

Output task C

A `.npy` file containing a NumPy array of size (N, 3) with the coordinates of all electrode contacts. Each row corresponds to one contact point, with three columns representing its voxel coordinates: i, j, and k.

	i	j	k
$coord_1$	[[45,	62,	31],
$coord_2$	[47,	64,	33],
$coord_3$	[90,	218,	135],
⋮		⋮	
$coord_n$	[424,	367,	134]]

The functions `np.save(filename, array)` and `np.load(filename)` allow you to save a NumPy array to a `.npy` file and load it back, respectively.

Bonus task
Assign a unique label to each electrode and associate each extracted contact point coordinate with the corresponding electrode label.

Output task C with bonus

Add a final column to the `.npy` file generated in task C, containing a label identifying the electrode m to which each contact point belongs. All contact points belonging to the same electrode should share the same label.

	i	j	k	$label$
$coord_1$	[[45,	62,	31,	0],
$coord_2$	[47,	64,	33,	0],
$coord_3$	[90,	218,	135,	1],
⋮		⋮		
$coord_n$	[424,	367,	134,	m]]

4 Code and report

As mentioned above, we provide you with a Python code containing several useful functions for handling the data. You are free to organize your work as you see fit (e.g., using Python scripts (`.py` files), Jupyter notebooks, etc.). Regardless of the approach you choose, all the code you write and use to complete the tasks must be submitted inside a single zipped folder.

The overall presentation, organization, and formatting of your code will also be taken into account in the assessment. Make sure to write clean, efficient, and well-structured code, with appropriate comments to make it easy to understand. The code must be executable within its own folder. To do so, use relative paths for the file names, i.e., a file path that points to a location based on your current working directory, rather than the root of the file system.

In addition, you are required to submit a PDF report explaining and discussing your results. In particular, you will have noticed that no ground truth is available against which to compare your results. You are therefore expected to critically assess the quality of your results, discuss any limitations of your approach, and identify potential improvements that could be considered. Any figures you generate should also be clear and well-presented. Use descriptive file names and, for figures, make sure to include appropriate titles, axis labels, legends, etc. Maximum one page per task, without taking into account the figures.

Here are some questions to consider when discussing the results of the 3 tasks :

- Under which conditions does your method perform well or fail ? Provide specific

examples and discuss the possible reasons.

- What are the limitations of your methods ? What additional processing steps or alternative approaches could be considered ?
- How well does your method generalize across different subjects ? Does it perform consistently ?
- Do the results obtained in one task influence the results of subsequent tasks ?

5 Submission

The assignment is due on **16th December 2026**. All files must be submitted into a single `.zip` folder on Moodle. It must include :

- Your code (`.py` or `.ipynb`)
- Your report (a `.pdf` file)
- For each subject, the outputs of each task (2 `.nii.gz` files and 1 `.npy` file):

Folder architecture

```
GROUP-X_NAME1_NAME2_NAME3.zip
└ data
   └ sub-01_CT.nii.gz
   └ sub-02_CT.nii.gz
   └ sub-03_CT.nii.gz
└ output
   └ sub-01_task-1_segmentation.nii.gz
   └ sub-01_task-2_brain-mask.nii.gz
   └ sub-01_task-3_coordinates.npy
   └ sub-02_task-1_segmentation.nii.gz
   └ sub-02_task-2_brain-mask.nii.gz
   └ sub-02_task-3_coordinates.npy
   └ sub-03_task-1_segmentation.nii.gz
   └ sub-03_task-2_brain-mask.nii.gz
   └ sub-03_task-3_coordinates.npy
└ GROUP-X_report.pdf
└ ...  .py/.ipynb
```